

Министерство образования и науки
федеральное государственное автономное образовательное учреждение
высшего образования
«Национальный исследовательский университет ИТМО»

Факультет инфокоммуникационных технологий

Отчет по дисциплине: «**Web-программирование**»
Практическая работа 6

Выполнила:

Гриндий Е.А.

Группа:

К3322

Проверила:

Марченко Е.В.

Санкт-Петербург,

2025

Цель: изучение работы с сокетами в Python, включая создание клиент-серверных приложений для: вычисления математических операций (например, калькулятор), передачи HTML-страниц (простой веб-сервер), реализации многопользовательского чата (обмен сообщениями в реальном времени).

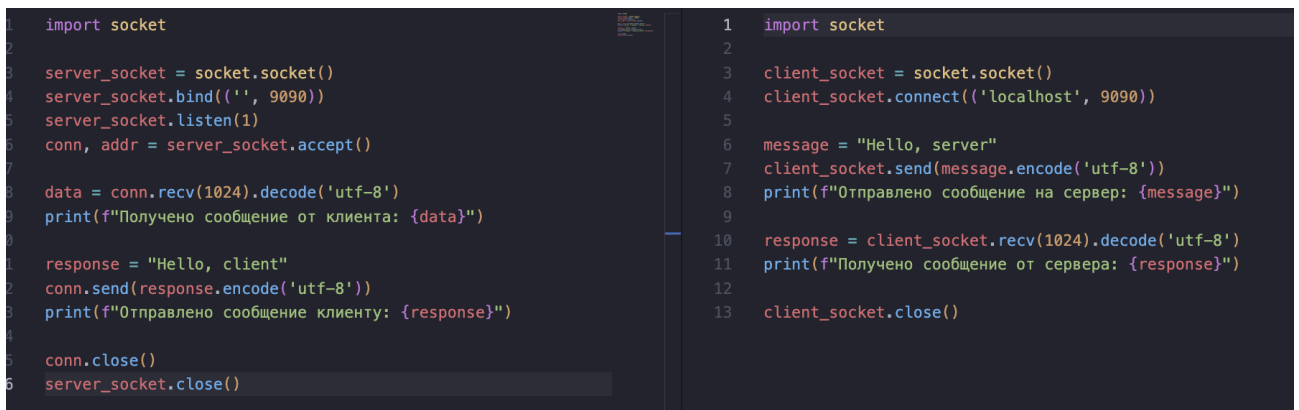
Задачи:

- Реализовать клиент-серверное приложение.
- Реализовать клиент-серверное приложение для выполнения математических операций.
- Реализовать серверную часть приложения, возвращающую HTML-страницу.
- Реализовать многопользовательский чат с использованием многопоточности.

Ход работы

Задание 1

Обмен сообщениями между клиентом и сервером (рисунок 1). Клиент отправляет сообщение "Hello, server", сервер получает его и отправляет ответ "Hello, client".



```
1 import socket
2
3 server_socket = socket.socket()
4 server_socket.bind(('', 9090))
5 server_socket.listen(1)
6 conn, addr = server_socket.accept()
7
8 data = conn.recv(1024).decode('utf-8')
9 print(f"Получено сообщение от клиента: {data}")
10
11 response = "Hello, client"
12 conn.send(response.encode('utf-8'))
13 print(f"Отправлено сообщение клиенту: {response}")
14
15 conn.close()
16 server_socket.close()
```

```
1 import socket
2
3 client_socket = socket.socket()
4 client_socket.connect(('localhost', 9090))
5
6 message = "Hello, server"
7 client_socket.send(message.encode('utf-8'))
8 print(f"Отправлено сообщение на сервер: {message}")
9
10 response = client_socket.recv(1024).decode('utf-8')
11 print(f"Получено сообщение от сервера: {response}")
12
13 client_socket.close()
```

Рисунок 1 - код программы

Результат работы программы представлен на рисунке 2.

```

garripotter@MacBook-Air-Garri task 1 % python3 server.py
Получено сообщение от клиента: Hello, server
Отправлено сообщение клиенту: Hello, client
garripotter@MacBook-Air-Garri task 1 %

```

Рисунок 2 - результат работы программы

Задание 2

Выполнение математических операций (решение квадратного уравнения) (рисунок 3). Клиент отправляет коэффициенты квадратного уравнения на сервер. Сервер вычисляет корни уравнения и возвращает результат клиенту. Результат корректно отображается на стороне клиента.

```

works > K3322 > Гриндий_Екатерина > lab_6 > task2 > server.py > solve_quadratic_equ
1 import socket
2 import math
3
4 def solve_quadratic_equation(a, b, c):
5     discriminant = b**2 - 4*a*c
6     if discriminant > 0:
7         x1 = (-b + math.sqrt(discriminant)) / (2*a)
8         x2 = (-b - math.sqrt(discriminant)) / (2*a)
9         return f"Корни уравнения: x1 = {x1}, x2 = {x2}"
10    elif discriminant == 0:
11        x = -b / (2*a)
12        return f"Уравнение имеет один корень: x = {x}"
13    else:
14        return "Уравнение не имеет действительных корней"
15
16 server_socket = socket.socket()
17 server_socket.bind(('localhost', 9090))
18 server_socket.listen(1)
19
20 while True:
21     client_socket, addr = server_socket.accept()
22
23     data = client_socket.recv(1024).decode()
24     a, b, c = map(float, data.split(','))
25     result = solve_quadratic_equation(a, b, c)
26
27     client_socket.send(result.encode())
28     client_socket.close()

```

```

works > K3322 > Гриндий_Екатерина > lab_6 > task2 > client.py > ...
1 import socket
2
3 a = float(input("Введите коэффициент a: "))
4 b = float(input("Введите коэффициент b: "))
5 c = float(input("Введите коэффициент c: "))
6
7 client_socket = socket.socket()
8 client_socket.connect(('localhost', 9090))
9
10 data = f"{a},{b},{c}"
11 client_socket.send(data.encode())
12
13 result = client_socket.recv(1024).decode()
14 print(f"Результат: {result}")
15
16 client_socket.close()

```

Рисунок 3 - код программы

Результат работы программы представлен на рисунке 4.

```

garripotter@MacBook-Air-Garri task2 % python3 client.py
Введите коэффициент a: 2
Введите коэффициент b: 5
Введите коэффициент c: 7
Результат: Уравнение не имеет действительных корней
garripotter@MacBook-Air-Garri task2 % python3 client.py
Введите коэффициент a: 1
Введите коэффициент b: 3
Введите коэффициент c: 2
Результат: Корни уравнения: x1 = -1.0, x2 = -2.0

```

Рисунок 4 - результат работы

Задание 3

Возврат HTML-страницы сервером (рисунок 5). Сервер возвращает HTML-страницу из файла index.html. Клиент (браузер) корректно отображает страницу по адресу <http://localhost:8080>.

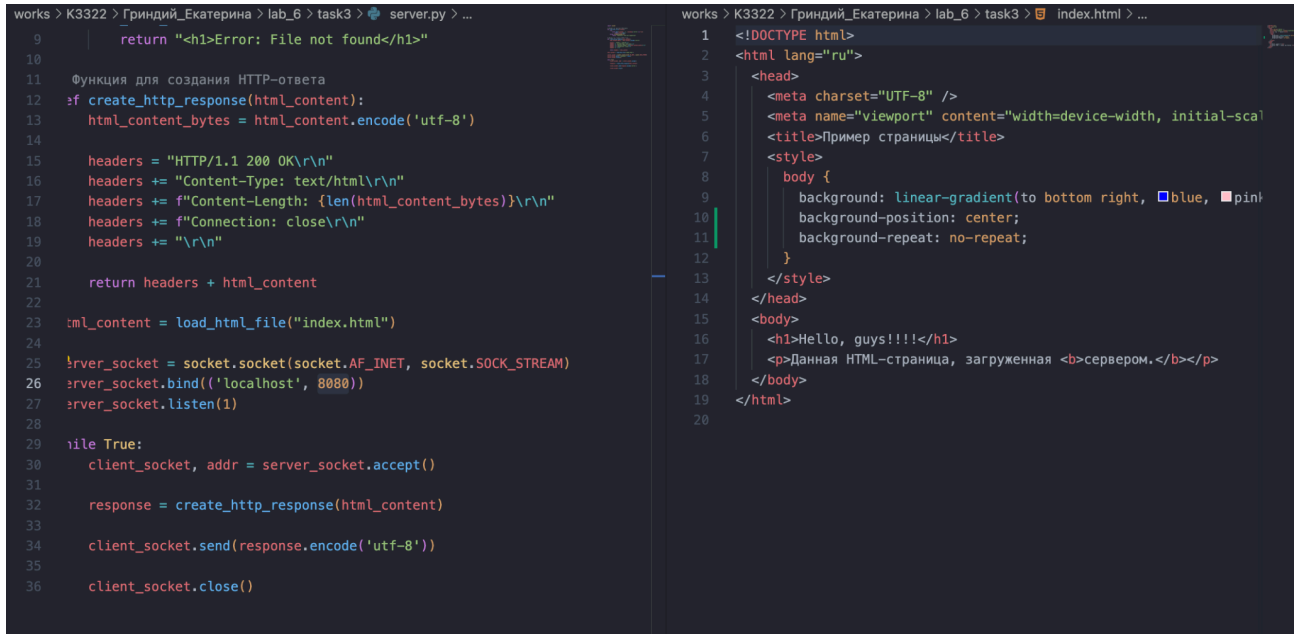


Рисунок 5 - код программы

Результат работы программы представлен на рисунке 6.

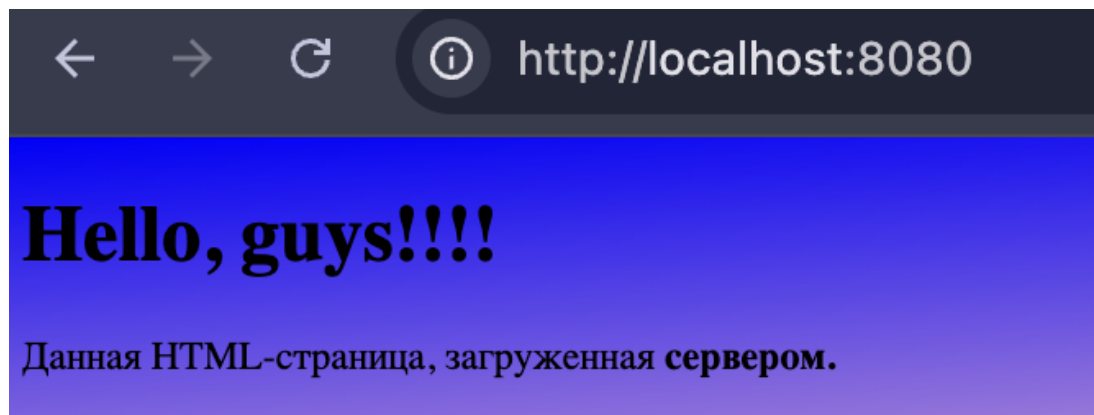


Рисунок 6 - результат работы

Задание 4

Многопользовательский чат (рисунок 7). Несколько клиентов могут подключаться к серверу и обмениваться сообщениями. Сообщения отображаются у всех участников чата. При подключении и отключении клиентов сервер уведомляет остальных участников.

```

import socket
import threading

clients = {}

def handle_client(client_socket, client_address):
    print(f"Подключен клиент: {client_address}")

    username = client_socket.recv(1024).decode('utf-8')
    clients[client_socket] = username
    print(f"Клиент {client_address} представился как {username}")

    broadcast(f"{username} присоединился к чату!", client_socket)

    while True:
        try:
            message = client_socket.recv(1024).decode('utf-8')
            if not message:
                break

            print(f"Получено сообщение от {username}: {message}")

            broadcast(f"{username}: {message}", client_socket)
        except Exception as e:
            print(f"Ошибка при обработке сообщения от {username}: {e}")
            break

    del clients[client_socket]
    client_socket.close()
    print(f"Клиент {username} отключен")

    broadcast(f"{username} покинул чат.", client_socket)

def broadcast(message, sender_socket=None):
    for client_socket in clients:
        if client_socket != sender_socket:
            try:
                client_socket.send(message.encode('utf-8'))
            except Exception as e:
                print(f"Ошибка при отправке сообщения клиенту: {e}")

server_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
server_socket.bind(('localhost', 9090))
server_socket.listen(5)

while True:
    client_socket, client_address = server_socket.accept()
    threading.Thread(target=handle_client, args=(client_socket, client_address)).start()

```

```

1 import socket
2 import threading
3
4 def receive_messages(client_socket):
5     while True:
6         try:
7             message = client_socket.recv(1024).decode('utf-8')
8             print(message)
9             except Exception as e:
10                print(f"Ошибка при получении сообщения: {e}")
11                break
12
13 client_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
14 client_socket.connect(('localhost', 9090))
15
16 username = input("Введите ваше имя: ")
17 client_socket.send(username.encode('utf-8'))
18
19 threading.Thread(target=receive_messages, args=(client_socket,)).start()
20
21 while True:
22     message = input()
23     if message.lower() == 'exit':
24         break
25     client_socket.send(message.encode('utf-8'))
26
27 client_socket.close()

```

Рисунок 7 - код программы

Результат работы программы представлен на рисунке 8.

```

garripotter@MacBook-Air-Garri task4 % python3 server.py
Подключен клиент: ('127.0.0.1', 58851)
Клиент ('127.0.0.1', 58851) представился как kate
Получено сообщение от kate: hello
Подключен клиент: ('127.0.0.1', 58980)
Клиент ('127.0.0.1', 58980) представился как aaron
Получено сообщение от aaron: hello kate

```

Рисунок 8 - результат работы

Вывод: в ходе лабораторной работы были изучены основы работы с сокетами в Python. Были разработаны клиент-серверные приложения для обмена сообщениями, выполнения математических операций, передачи HTML-страниц и организации многопользовательского чата. Все задачи выполнены успешно, приложения работают корректно.